

Lab 9.1 – Documentation Generation -Automatic documentation and code comments

Name: K.Karthik

Roll no: 2503A52420

Batch: 14

Task Description :1 – **Docstring , Inline comments , Google-style documentation**

Task: Compare the three approaches. Discuss their advantages and disadvantages. Explain suitable use cases. Recommend the best documentation style for a **mathematical utilities library**

Prompt: Explain what the function does, describe the input parameter, and describe the return value. Keep the documentation simple and suitable for a beginner.

Input:

```
def find_max_docstring(numbers):  
    """Return the maximum number from a list of numbers."""  
    return max(numbers)
```

```
def find_max_inline(numbers):
```

```
# Check and return the maximum value in the input list
return max(numbers) # Built-in max function handles the
comparison
```

```
def find_max_google(numbers):
```

```
    """Finds the maximum value in a list of numbers.
```

```
    Args:
```

```
        numbers (list of int/float): A list of numerical values to
search.
```

```
    Returns:
```

```
        int/float: The largest number in the list.
```

```
    Raises:
```

```
        ValueError: If the input list is empty.
```

```
    """
```

```
    if not numbers:
```

```
        raise ValueError("The list of numbers cannot be empty.")
```

```
    return max(numbers)
```

#example usage

```
numbers = [3, 5, 1, 8, 2]
```

```
print(find_max_docstring(numbers)) # Output: 8
```

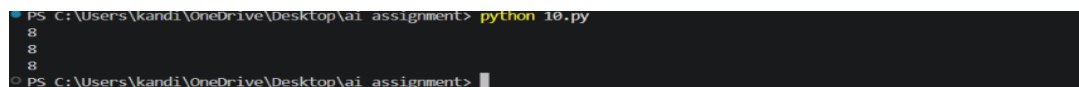
```
print(find_max_inline(numbers)) # Output: 8
```

```
print(find_max_google(numbers)) # Output: 8
```

Explanation:

The purpose of this problem is to understand **different ways of documenting Python code**.

Output:

A screenshot of a Windows PowerShell terminal window. The prompt is 'PS C:\Users\kandi\OneDrive\Desktop\ai_assignment>'. The user has entered 'python 10.py'. The output of the script is displayed on the next line, showing the number '8' three times, each on a new line. The terminal background is dark, and the text is light-colored.

```
PS C:\Users\kandi\OneDrive\Desktop\ai_assignment> python 10.py
8
8
8
PS C:\Users\kandi\OneDrive\Desktop\ai_assignment>
```

Task Description :2 – Documentation for login()

Task: Write documentation using all three formats. Critically compare the approaches. Recommend which documentation style would be most useful for **new developers joining a project**.

Prompt : Generate dd a clear inline comment to the following Python function. The comment should explain what the function is doing without unnecessarily explaining obvious Python syntax.

```
def find_max(numbers):
    return max(numbers)
```

Input:

```
def login_docstring(user, password, credentials):  
    """Verify if the provided password matches the stored  
    password for the given user in the credentials registry."""  
    return credentials.get(user) == password  
  
def login_inline(user, password, credentials):  
    # Retrieve password for user and evaluate match  
    # Using .get() prevents KeyError if user is missing, returning  
    None which safely compares False  
    return credentials.get(user) == password  
  
def login_google(user, password, credentials):  
    """Authenticates a user against a provided credentials store.
```

Args:

user (str): The unique identifier/username of the user trying to log in.

password (str): The plain-text security password provided by the client.

credentials (dict): A dictionary mapping usernames (str) to their corresponding correct passwords (str).

Returns:

bool: True if the user exists and the password matches; False otherwise.

"""

return credentials.get(user) == password

#example usage

```
credentials = {'alice': 'password123', 'bob': 'securepass'}
```

```
print(login_docstring('alice', 'password123', credentials)) #
```

Output: True

```
print(login_inline('alice', 'password123', credentials)) #
```

Output: True

```
print(login_google('alice', 'password123', credentials)) #
```

Output: True

Explanation:

This problem is similar to Problem 1, but the function represents a **login/authentication operation**.

Output:

```
PS C:\Users\kandi\OneDrive\Desktop\ai_assignment> python 10.py
True
True
True
```

Task Description :3 – Calculator Module

Task: Display the module documentation in the terminal.
Generate the documentation as an **HTML file** using Python's pydoc. Open the generated HTML file in a browser and verify it methods.

Prompt: Generate Convert the following Python function into a properly documented Google-style function. Include a description, Args section, and Returns section.

```
def find_max(numbers):
    return max(numbers)
```

Input:

```
"""A basic calculator module providing arithmetic operations
with structured documentation.
```

```
"""
```

```
def add(a, b):
```

"""Returns the sum of two numbers.

Args:

a (int/float): The first number.

b (int/float): The second number.

Returns:

int/float: The sum of a and b.

"""

return a + b

def subtract(a, b):

"""Returns the difference of two numbers.

Args:

a (int/float): The first number (minuend).

b (int/float): The second number (subtrahend).

Returns:

int/float: The difference of a and b.

"""

return a - b

def multiply(a, b):

"""Returns the product of two numbers.

Args:

a (int/float): The first factor.

b (int/float): The second factor.

Returns:

int/float: The product of a and b.

"""

return a * b

def divide(a, b):

"""Returns the quotient of two numbers.

Args:

a (int/float): The dividend.

b (int/float): The divisor.

Returns:

float: The quotient of a divided by b.

Raises:

ZeroDivisionError: If the divisor b is zero.

"""

if b == 0:

raise ZeroDivisionError("Division by zero is not allowed.")

return a / b

#example usage

print(divide(10, 2)) # Output: 5.0

print(multiply(3, 4)) # Output: 12

print(subtract(10, 5)) # Output: 5

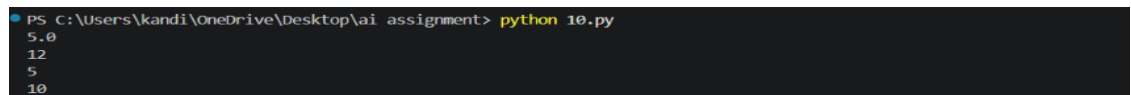
print(add(7, 3)) # Output: 10

Explanation:

This problem introduces **automatic documentation generation**.

Instead of manually creating a documentation website, Python's `pydoc` can read the docstrings from your program and generate documentation.

Output:

A terminal window with a dark background. The prompt is 'PS C:\Users\kandi\OneDrive\Desktop\ai_assignment>'. The command 'python 10.py' has been executed. The output is displayed on the next line: '5.0', '12', '5', and '10' on separate lines.

```
PS C:\Users\kandi\OneDrive\Desktop\ai_assignment> python 10.py
5.0
12
5
10
```

Task Description :4 – Conversion Utilities Module

Task: Use **Copilot** to automatically generate docstrings.

Generate the documentation in the terminal. Export the documentation to HTML. Open the HTML documentation in a browser.

Prompt: Generate Compare docstrings, inline comments, and Google-style documentation for the following Python function:

```
def find_max(numbers):
    return max(numbers)
```

For each approach, explain:

1. Advantages
2. Disadvantages

3. Suitable use cases

Finally, recommend the most effective documentation style for a mathematical utilities library and justify your answer.

Input:

```
"""A conversion utility module providing functions to convert
numbers between decimal, binary, and hexadecimal bases.
"""
```

```
def decimal_to_binary(n):
```

```
    """Converts a decimal integer to its binary string
representation.
```

Args:

n (int): The decimal integer to convert.

Returns:

str: The binary representation of the integer (prefixed with '0b').

Raises:

TypeError: If the input is not an integer.

```
"""
```

```
if not isinstance(n, int):
```

```
    raise TypeError("Input must be an integer.")
```

```
return bin(n)
```

```
def binary_to_decimal(b):
```

```
    """Converts a binary string to its decimal integer
    representation.
```

Args:

b (str): The binary string to convert (can optionally start with '0b').

Returns:

int: The decimal integer value of the binary string.

Raises:

ValueError: If the input string is not a valid binary number.

TypeError: If the input is not a string.

```
"""
```

```
if not isinstance(b, str):
```

```
    raise TypeError("Input must be a string.")
```

```
return int(b, 2)
```

```
def decimal_to_hexadecimal(n):
```

```
    """Converts a decimal integer to its hexadecimal string
    representation.
```

```
    Args:
```

```
        n (int): The decimal integer to convert.
```

```
    Returns:
```

```
        str: The lowercase hexadecimal representation of the
        integer (prefixed with '0x').
```

```
    Raises:
```

```
        TypeError: If the input is not an integer.
```

```
"""
```

```
if not isinstance(n, int):
```

```
        raise TypeError("Input must be an integer.")
    return hex(n)

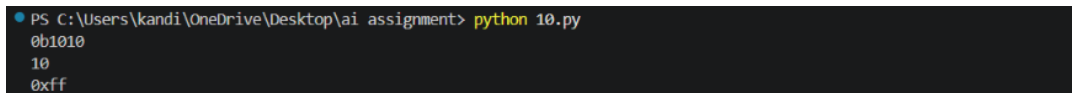
#example usage

print(decimal_to_binary(10)) # Output: '0b1010'
print(binary_to_decimal('0b1010')) # Output: 10
print(decimal_to_hexadecimal(255)) # Output: '0xff'
```

Explanation:

This problem combines programming + AI-assisted documentation + automatic documentation generation.

Output:

A terminal window with a dark background. The prompt is 'PS C:\Users\kandi\OneDrive\Desktop\ai_assignment>'. The command 'python 10.py' has been executed. The output is displayed on three lines: '0b1010', '10', and '0xff'.

```
PS C:\Users\kandi\OneDrive\Desktop\ai_assignment> python 10.py
0b1010
10
0xff
```

Task-5: Course Management Module

Prompt: Generate a Python module named course.py for a simple course management system.

```
"""A course management module providing utilities to add,
remove, and retrieve course details.
```

```
"""
```

```
# In-memory database to store course information
```

```
_course_db = {}
```

```
def add_course(course_id, name, credits):
```

```
    """Adds a new course to the registry.
```

```
    Args:
```

```
        course_id (str): The unique identifier for the course (e.g.,  
'CS101').
```

```
        name (str): The name of the course.
```

```
        credits (int): The number of academic credits for the  
course.
```

```
    Raises:
```

```
        ValueError: If the course_id already exists in the registry.
```

```
        TypeError: If credits is not an integer or if course_id/name  
is not a string.
```

```
    """
```

```
    if not isinstance(course_id, str) or not isinstance(name, str):
```

```
        raise TypeError("course_id and name must be strings.")
```

```
    if not isinstance(credits, int):
```

```
        raise TypeError("credits must be an integer.")
    if course_id in _course_db:
        raise ValueError(f"Course with ID {course_id} already
exists.")
```

```
_course_db[course_id] = {
    "name": name,
    "credits": credits
}
```

```
def remove_course(course_id):
```

```
    """Removes an existing course from the registry.
```

Args:

course_id (str): The unique identifier of the course to be removed.

Raises:

KeyError: If the course_id is not found in the registry.

```
    """
```

```
if course_id not in _course_db:
    raise KeyError(f"Course with ID {course_id} does not
exist.")
del _course_db[course_id]
```

```
def get_course(course_id):
    """Retrieves the details of a specific course.
```

Args:

course_id (str): The unique identifier of the course to retrieve.

Returns:

dict: A dictionary containing the course 'name' and 'credits'.

Raises:

KeyError: If the course_id is not found in the registry.

```
"""
```

```
if course_id not in _course_db:
```

```
    raise KeyError(f"Course with ID {course_id} does not  
exist.")  
  
    return _course_db[course_id]  
  
#example usage  
  
add_course('CS101', 'Introduction to Computer Science', 4)  
  
print(get_course('CS101')) # Output: {'name': 'Introduction to  
Computer Science', 'credits': 4}
```

Explanation:

Review the course.py module and check whether the documentation for add_course(), remove_course(), and get_course() is complete and consistent.

Output:

```
PS C:\Users\kandi\OneDrive\Desktop\ai assignment> python course.py  
{'name': 'Introduction to Computer Science', 'credits': 4}
```